



Conceptos básicos de orientación a objeto en python

Clase

Codificar una rutina utilizando clases provistas para la resolución de un problema simple de acuerdo al paradigma de orientación a objetos en el entorno python.

- Unidad 1: El lenguaje Python
- Unidad 2: Sentencias básicas del lenguaje Python
- Unidad 3: Sentencias condicionales
- Unidad 4: Funciones y módulos
- Unidad 5: Estructura de datos en Python
- Unidad 6: Secuencias iterativas
- Unidad 7: Conceptos básicos de orientación a objeto en Python



Te encuentras aquí



¿Qué aprenderás en esta sesión?

- *Explica los elementos fundamentales del paradigma de orientación a objetos en la programación.*
- *Distingue la diferencia entre una clase y un objeto, en el contexto de la programación orientada a objetos.*
- *Implementa una clase utilizando un método inicializador con parámetros y métodos personalizados para resolver un problema de bajísima complejidad.*
- *Utiliza clases y métodos existentes en librerías para la resolución de un problema de baja complejidad.*

Piensa en tu videojuego favorito. En él probablemente hay personajes, objetos, vehículos o elementos que interactúan entre sí. Cada uno tiene características propias (color, tamaño, poder) y puede realizar diferentes acciones (saltar, atacar, defenderse). ¿Cómo organizarías todas estas características y acciones si tuvieras que programar el juego? ¿Agruparías elementos similares de alguna manera?"



/* Paradigma de orientación a objetos en Python */

¿Qué es un Paradigma de Programación? 🤔

Un paradigma de programación es como una "forma de pensar" cuando programamos. Es similar a los diferentes estilos que existen para organizar una biblioteca:

- Por género (terror, romance, etc.).
- Por autor (alfabéticamente).
- Por fecha de publicación.

La P00: Una Forma Diferente de Ver el Código 🌟

La Programación Orientada a Objetos (P00) es una forma de organizar y escribir código que se basa en estructurar todo como "objetos" del mundo real.

Piensa en una Fábrica de Autos 🚗

- **Sin P00** sería como tener todas las piezas y características de los autos dispersas:
 - Una lista de colores
 - Otra lista de marcas
 - Otra de modelos

{desafío} Y así sucesivamente...
latam_

- **Con P00** es como tener un plano (clase) que dice:
 - "Un auto tiene: color, marca, modelo".
 - "Un auto puede: arrancar, frenar, acelerar".
 - Y cada auto que fabricas (objeto) sigue ese plano.

Un Ejemplo más Cercano: Un Colegio

En un colegio tienes estudiantes. Cada estudiante tiene:

- Características (nombre, edad, curso).
- Cosas que puede hacer (estudiar, rendir pruebas, asistir a clase).

En POO diríamos:

1. La "clase" Estudiante es como una plantilla que define qué es ser un estudiante.
2. Cada estudiante real es un "objeto" creado desde esa plantilla.
3. Todos los estudiantes tienen las mismas características y pueden hacer las mismas cosas, pero con sus propios valores.

¿Para qué sirve esto? 🤔

Organización:

- Es como tener una caja para cada tipo de cosa.
- Todo lo relacionado está junto.
- Es más fácil encontrar y modificar las cosas.

Reutilización:

- No tienes que escribir las mismas cosas muchas veces.
- Creas un modelo una vez y lo usas cuando quieras.

Mantenimiento:

- Si necesitas cambiar algo, lo cambias en un solo lugar.
- Todos los objetos se actualizan automáticamente.

Ahora sí, veamos un ejemplo con código, paso a paso:

```
# 1. Primero, creamos el "plano" de lo que es un estudiante
class Estudiante:
    # Este es el "molde" que dice qué tiene y qué hace un estudiante

    # Cuando creamos un nuevo estudiante, necesita estos datos:
    def __init__(self, nombre, edad):
        self.nombre = nombre # Guardamos su nombre
        self.edad = edad     # Guardamos su edad
        self.notas = []      # Comienza sin notas
```

¿Qué significa esto?

- `class Estudiante`: Estamos creando un plano/molde.
- `__init__`: Es como decir "cuando crees un nuevo estudiante, haz esto".
- `self`: Es una referencia al estudiante específico que estamos creando.
- `self.nombre = nombre`: Guardamos el nombre que nos dan.

Ahora, ¿cómo lo usamos?

```
# Creamos un estudiante nuevo:
ana = Estudiante("Ana", 15)

# ¿Qué significa esto?
# 1. Python mira el "plano" Estudiante
# 2. Crea un nuevo objeto siguiendo ese plano
# 3. Le asigna el nombre "Ana" y edad 15
# 4. Lo guarda en la variable 'ana'|
```

Es como si dijéramos:

1. Tengo un formulario (clase) que define qué información necesito de un estudiante.
2. Cada vez que lleno ese formulario, creo un nuevo estudiante (objeto).
3. Cada estudiante tiene su propia información, pero todos siguen el mismo formato.

Principios básicos de la P00

Imagina que estás construyendo una casa desde cero. Para hacerlo necesitas:

- Un plano (Clases).
- Materiales y características específicas (Atributos).
- Diferentes funcionalidades en cada habitación (Métodos).
- Reglas de construcción (Principios).

La P00 funciona de manera similar: nos da las herramientas y principios para construir programas de manera organizada y eficiente.

Principios básicos de la P00

Clases y objetos

- **Clase:** Es un plano o plantilla que define las características y comportamientos que tendrá un tipo de objeto.
- **Objeto:** Es una instancia específica creada a partir de una clase.

Piensa en una fábrica de galletas:

- La **clase** es el molde para galletas
- Las **galletas** son los objetos
- Cada galleta sigue el mismo molde pero puede tener diferentes sabores o decoraciones

Principios básicos de la P00

Clases y objetos

```
# Definición de una clase
class Perro:
    def __init__(self, nombre, raza):
        self.nombre = nombre
        self.raza = raza

# Creación de objetos
bobby = Perro("Bobby", "Labrador")
luna = Perro("Luna", "Husky")
```

Principios básicos de la P00

Abstracción y ocultamiento

Abstracción: Es simplificar algo complejo ocultando los detalles innecesarios y mostrando solo lo esencial.

Ocultamiento: Es esconder los detalles internos de cómo funciona algo, mostrando solo lo que se necesita usar.

Un control remoto de TV:

- **Abstracción:** Solo ves los botones necesarios
- **Ocultamiento:** No necesitas saber cómo funciona por dentro

Principios básicos de la P00

Abstracción y ocultamiento

```
class SmartTV:
    def __init__(self):
        self.__estado = "apagado" # Atributo privado
        self.__canal = 1          # Atributo privado

    def encender(self):            # Método público
        self.__estado = "encendido"
        print("TV encendida")

    def cambiar_canal(self, canal): # Método público
        if self.__estado == "encendido":
            self.__canal = canal
            print(f"Canal cambiado a: {canal}")
        else:
            print("La TV está apagada")
```


Principios básicos de la P00

Estado y atributos

Estado: Es la información que un objeto mantiene en un momento dado.

Atributos: Son las variables que almacenan el estado de un objeto.

Piensa en un videojuego:

- El **estado** de un personaje incluye:
 - Vida actual
 - Nivel
 - Posición
- Estos datos se guardan en **atributos**

Principios básicos de la P00

Estado y atributos

```
class PersonajeJuego:
    def __init__(self):
        # Estados almacenados en atributos
        self.vida = 100
        self.nivel = 1
        self.posicion_x = 0
        self.posicion_y = 0
        self.nombre = "Jugador"
```

Principios básicos de la P00

Comportamiento y métodos

Comportamiento: Son las acciones que un objeto puede realizar.

Métodos: Son las funciones que definen el comportamiento de un objeto.

Un robot de cocina:

- **Comportamientos:** Mezclar, picar, batir
- Estos se implementan como **métodos** en la programación

Principios básicos de la P00

Comportamiento y métodos

```
class RobotCocina:
    def __init__(self):
        self.encendido = False
        self.velocidad = 0

    def mezclar(self, velocidad): # Método
        self.velocidad = velocidad
        print(f"Mezclando a velocidad {velocidad}")

    def picar(self): # Método
        print("Picando ingredientes")

    def detener(self): # Método
        self.velocidad = 0
        print("Robot detenido")
```

Constructores e instanciación

Constructor: Es un método especial (**init** en Python) que se ejecuta cuando se crea un nuevo objeto.

Instanciación: Es el proceso de crear un objeto a partir de una clase.

Es como armar un mueble nuevo:

- El **constructor** es el manual de instrucciones
- La **instanciación** es el proceso de armado
- El **mueble terminado** es el objeto

Constructores e instanciación

```
class Estudiante:
    # Constructor
    def __init__(self, nombre, edad):
        # Inicialización de atributos
        self.nombre = nombre
        self.edad = edad
        self.notas = []

    def agregar_nota(self, nota):
        self.notas.append(nota)

# Instanciación
ana = Estudiante("Ana", 15) # Se ejecuta el constructor
juan = Estudiante("Juan", 16) # Otra instancia
```

Principios básicos de la P00

¿Cómo se relacionan entre sí?

Estos principios no funcionan de manera aislada, sino que se complementan:

```
# Ejemplo integrado: Una Smart Home
class CasaInteligente:
    def __init__(self, direccion):          # Constructor
        self.__direccion = direccion      # Atributo (oculto)
        self.luces = False                # Estado
        self.temperatura = 20              # Estado

    def ajustar_temperatura(self, temp):    # Método/Comportamiento
        self.temperatura = temp
        print(f"Temperatura ajustada a {temp}°C")
```

`/* El objeto String */`

El objeto string

Creación de strings

Un string en Python es una secuencia inmutable de caracteres. Esto significa que una vez creado, no puedes modificar sus caracteres individuales.

Formas de crear strings:

- Con comillas simples: 'texto'
- Con comillas dobles: "texto"
- Con triple comillas: """texto multilinea"""

```
# Diferentes formas de crear strings
nombre = "Ana"           # Con comillas dobles
apellido = 'González'    # Con comillas simples
frase = """Este es un    # Con triples comillas
texto de múltiples
líneas"""
```

El objeto string

Métodos principales de strings

Métodos más utilizados:

- `upper()` y `lower()`: Convertir a mayúsculas/minúsculas.
- `strip()`: Eliminar espacios al inicio y final.
- `replace()`: Reemplazar texto.
- `split()`: Dividir el string en una lista.
- `join()`: Unir elementos de una lista en un string.

Operaciones básicas:

- Concatenación con `+`
- Repetición con `*`
- Formateo con f-strings (la forma más moderna)

Acceso a caracteres:

- Por índice positivo: `texto[0]` (primer carácter)
- Por índice negativo: `texto[-1]` (último carácter)
- Por slicing: `texto[inicio:fin:paso]`

**¡Manos a la obra! Aplicando
lo aprendido**



Aplicando lo aprendido

1. Descarga el archivo POO.ipynb
2. Realiza cada una de las actividades que se solicitan.



Resumen

Subtítulo

- La POO es un paradigma que nos permite organizar el código pensando en "objetos" del mundo real
- Una clase es como un plano o plantilla que define qué características y comportamientos tendrá un tipo de objeto
- Los objetos son instancias específicas creadas a partir de una clase
- Los atributos son las características que definen el estado de un objeto
- Los métodos son las acciones que un objeto puede realizar
- El constructor (**init**) es un método especial que se ejecuta al crear un nuevo objeto
- La abstracción y el ocultamiento nos ayudan a simplificar y organizar nuestro código
- Los strings en Python son objetos que tienen sus propios métodos y características

Piensa en tres objetos cotidianos que uses regularmente. ¿Cómo podrías representarlos usando los conceptos de POO que aprendimos hoy? ¿Qué atributos y métodos definirías para cada uno? ¿Cómo te ayuda este ejercicio a comprender mejor la programación orientada a objetos?"





Próxima sesión...

Prueba.

{desafío}
latam_



{desafío}
latam_

*Academia de
talentos digitales*

